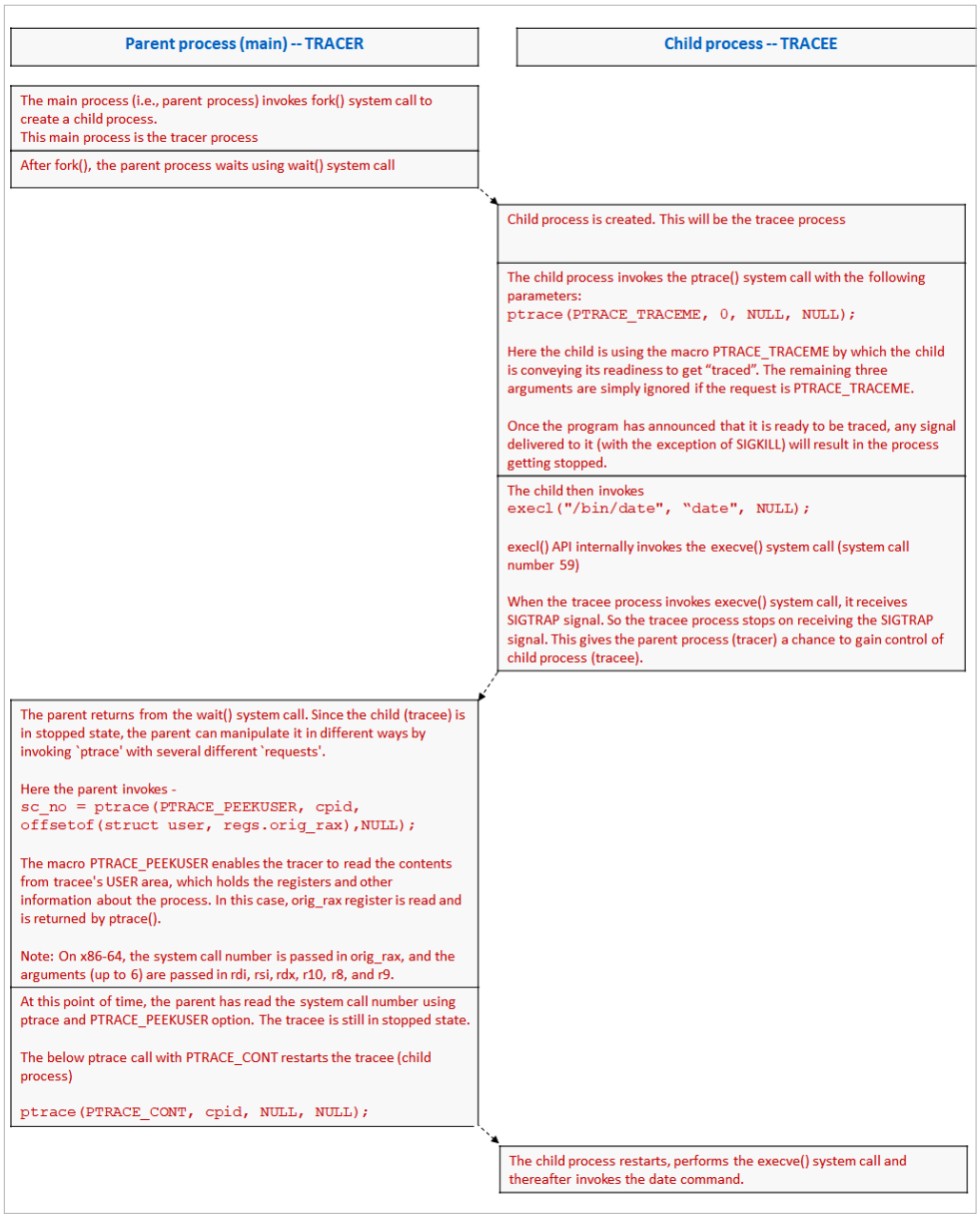

Figure 1: Detailed workflow of *mystrace.c* in x86_64 Linux environment

```
        switch(cpid = fork()){
                case -1:
                printf("fork() failure - exit from program");
                exit(1);

                case 0: /* in the child process */
                ptrace(PTRACE_TRACEME, 0, NULL, NULL);
                execl("/bin/date", "date", NULL);

                default: /* in the parent process */
                wait(NULL);
                sc_no = ptrace(PTRACE_PEEKUSER, cpid,
                offsetof(struct user, regs.
                orig_rax),NULL);

                printf("First system
                call(number) invoked by
                tracee is %ld\n", sc_no);

                ptrace(PTRACE_CONT, cpid,
                NULL, NULL);
                        }
        }

        $ gcc -o mystrace mystrace.c

        $ ./mystrace
        First system call(number)
        invoked by tracee is 59
        Wed Mar  8 03:00:02 JST 2023
```

When the *mystrace.c* program is compiled and executed, it shows that system call number 59 is the first system call that is invoked by the tracee (i.e., child process). System call number 59 is for *execve()* system call. The *execl()* API internally invokes the *execve()* system call.

Figure 1 explains the detailed sequence of steps performed by both tracer and tracee in an x86_64 Linux environment.

Ptrace is a powerful system call that provides a range of capabilities for observing and controlling the execution of another process. It can be used for both legitimate and malicious purposes. Attackers can use ptrace to inject malicious code into a running process or modify the behaviour of the process. As a result, ptrace is often disabled and restricted on production systems. END

By: Subranshu Patel

The author is a group manager in the system software group of HCL Technologies Limited. He has extensive experience in Linux-based OS and ecosystem design and development, Linux system software, and open source technologies.